

Sourcing CRM — IT Deployment Guide

Oct 8, 2026 · @Fardeen ismail

What IT is being asked to set up

The Sourcing CRM is one Node.js service with a single SQLite database file. It needs no database server, no message queue and no third-party packages. IT provides a Linux server inside the bank network, a DNS name with a TLS certificate, and the HTTPS reverse proxy in front. Installation takes about an hour.

Item	Requirement
Server	Linux VM or container, 2 vCPU, 2 GB RAM, 20 GB disk (database grows about 1 MB per 1,000 cases)
Runtime	Node.js 22.13 or newer, or Docker Engine 24+ with the compose plugin
Network	HTTPS from staff laptops and phones to the server; the server needs no outbound internet
DNS and TLS	A name such as <code>crm-test.yourbank.ae</code> with a certificate the bank's browsers trust
Source code	Git access to the <code>fardeen79-ops/crm</code> repository, branch <code>claude/sales-sourcing-crm-workflow-rdlw4h</code>
Storage	The database directory on backed-up, encrypted storage

HTTPS is mandatory, not optional. The sign-in cookie is marked Secure, and browsers only allow the camera (Emirates ID scanner) and microphone (voice typing) on HTTPS pages.

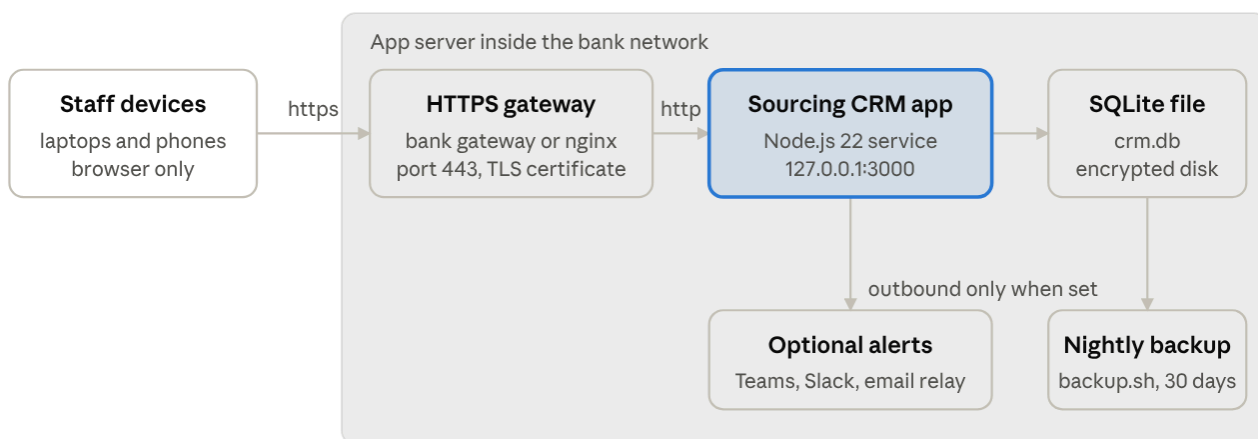
Who does what

Role	Responsibility
IT infrastructure	Server, DNS, certificate, reverse proxy, firewall, backups, updates (Steps 1 to 6)
Business owner (the person requesting this)	Names the first team leader, the MIS user, and signs off the acceptance test (Steps 5 and 8)

Role	Responsibility
MIS user	Bulk uploads of users, cases, targets and card activation inside the app
Information security	Reviews the security notes before any real customer data is entered

Architecture

One server runs the whole CRM behind the bank HTTPS gateway



deployment architecture · 4 components, 2 optional

Staff browsers reach the gateway on port 443 only; the gateway forwards to the app on the loopback port, and the app reads and writes one database file. The only outbound traffic is to alert webhooks, and only when IT configures them.

Step 1: Prepare the server

One Ubuntu 22.04 or 24.04 LTS server (or RHEL 9) inside the bank network, reachable from staff devices on port 443 only.

1. Create the server and give it the DNS name (for example `crm-test.yourbank.ae`). Confirm the name resolves from a staff laptop.
2. Obtain the TLS certificate and private key for that name from the bank's CA. Place them at `/etc/ssl/certs/<name>.crt` and `/etc/ssl/private/<name>.key`.
3. Firewall: allow inbound 443 (and 80 only for the redirect) from staff networks; allow 22 from the IT admin network. Block everything else inbound. No outbound internet is required.
4. Install git and either Node.js 22 or Docker:

```
# Node.js route (Ubuntu)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs git sqlite3
node --version          # must print v22.13.0 or newer

# Docker route
sudo apt-get install -y git docker.io docker-compose-v2
sudo systemctl enable --now docker
```

5. Node.js route only: create the service account and folders.

```
sudo useradd -r -s /usr/sbin/nologin crm
sudo mkdir -p /opt/sourcing-crm /var/lib/sourcing-crm /var/backups/sourcing-crm
```

6. Make sure `/var/lib/sourcing-crm` (or the Docker volume) is on encrypted disk and is included in the server backup policy.

7. Set the server clock to sync with NTP. Sales cycles and call-back times are computed in UAE time from the server clock.

Getting the code onto the server

The repository is on a personal GitHub account, which the bank server will usually not be allowed to reach. Transfer the code once as a file, then let the server work from a local copy. Pick one of three ways; in Step 2 replace the `git clone` and `git checkout` lines with the matching "on the server" commands below.

Option 1: Source archive (works for both routes, simplest)

On the personal machine, with Node.js 22 installed:

```
git clone https://github.com/fardeen79-ops/crm.git sourcing-crm
cd sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
npm run setup:ocr          # downloads the Emirates ID scanner
files (needs internet)
tar czf ../sourcing-crm.tar.gz --exclude=.git --exclude=data --exclude='*.db'
--exclude=.env .
sha256sum ../sourcing-crm.tar.gz      # note the checksum
```

Move `sourcing-crm.tar.gz` (about 15 MB) to the server by the bank's approved method: secure file transfer portal, SFTP from the admin network, or encrypted USB. Then on the server:

```
sha256sum /path/to/sourcing-crm.tar.gz # must match the checksum noted
above
sudo mkdir -p /opt/sourcing-crm
sudo tar xzf /path/to/sourcing-crm.tar.gz -C /opt/sourcing-crm
```

Skip `npm run setup:ocr` in Step 2; the scanner files are already in the archive. Updates arrive as a new archive: stop the service, extract over `/opt/sourcing-crm`, start the service. The database is in `/var/lib/sourcing-crm`, not in the code folder, so it is untouched.

Option 2: Push to the bank's internal Git (best for ongoing updates)

If IT runs GitLab, Bitbucket or Azure DevOps, create an empty project there and ask for the person's account to have push rights. On the personal machine:

```
git clone --mirror https://github.com/fardeen79-ops/crm.git
cd crm.git
git push --mirror https://git.yourbank.ae/sales/sourcing-crm.git
```

The server then follows Step 2 exactly, with the internal URL in place of the GitHub one. Future updates are a `git push` from the personal machine and a `git pull` on the server. The scanner files still need internet once (`npm run setup:ocr`), so run that on the personal machine and commit `public/vendor/tesseract` to the internal repository, or transfer that folder with Option 1.

Option 3: Docker image file (Docker route only)

Build the image on the personal machine and move the finished image, so the server needs neither GitHub nor the Docker Hub base image.

```
# personal machine
git clone https://github.com/fardeen79-ops/crm.git sourcing-crm && cd
sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
docker build -f deploy/Dockerfile -t sourcing-crm:1.0 .
docker save sourcing-crm:1.0 | gzip > sourcing-crm-1.0.tar.gz # about 70 MB
```

Transfer `sourcing-crm-1.0.tar.gz`, `deploy/docker-compose.yml` and `deploy/.env.example` to the server. On the server:

```
docker load < sourcing-crm-1.0.tar.gz
mkdir -p /opt/sourcing-crm/deploy && cp docker-compose.yml .env.example
/opt/sourcing-crm/deploy/
```

Edit `/opt/sourcing-crm/deploy/docker-compose.yml`: delete the `build:` block and its two lines under it, and add `image: sourcing-crm:1.0` in its place. Then run the `docker compose ... up -d` command from Step 2 without `--build`. Each update is a new image file with a new tag.

Whichever option is used, the transferred file contains code only, no customer data. Keep the checksum with the change record so IT can show what was installed.

Step 2: Install and start the app

Pick one route. Docker is simpler; systemd is for servers where Docker is not approved. Both end with the app listening on `127.0.0.1:3000`, reachable only by the reverse proxy on the same host.

Route A: Docker

```
git clone https://github.com/fardeen79-ops/crm.git /opt/sourcing-crm
cd /opt/sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
cp deploy/.env.example deploy/.env
nano deploy/.env                                # fill in: see Step 4
chmod 600 deploy/.env
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --
build
docker compose -f deploy/docker-compose.yml logs -f crm    # Ctrl-C when you
see "listening"
```

The database lives in the Docker volume `crm-data` (`docker volume inspect deploy_crm-data` shows its path on disk). The image is built from the repository; there is no download from a public registry other than the `node:22-alpine` base image, which IT can mirror internally if required.

Route B: systemd (no Docker)

```

sudo git clone https://github.com/fardeen79-ops/crm.git /opt/sourcing-crm
cd /opt/sourcing-crm
sudo git checkout claude/sales-sourcing-crm-workflow-rdlw4h
sudo npm run setup:ocr          # copies the Emirates ID scanner files
so they are served locally
sudo cp deploy/.env.example /etc/sourcing-crm.env
sudo nano /etc/sourcing-crm.env  # fill in: see Step 4
sudo chmod 600 /etc/sourcing-crm.env
sudo chown -R crm:crm /opt/sourcing-crm /var/lib/sourcing-crm
sudo cp deploy/sourcing-crm.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now sourcing-crm
sudo journalctl -u sourcing-crm -f  # Ctrl-C when you see "listening"

```

The unit file runs as the unprivileged `crm` user with `ProtectSystem=strict`, so the process can only write to `/var/lib/sourcing-crm`.

Check it is up (either route):

```
curl -s http://127.0.0.1:3000/api/meta | head -c 200
```

A JSON response means the app is running. The first start creates the database and the first team leader account from `ADMIN_EMAIL` and `ADMIN_PASSWORD`.

Step 3: HTTPS and the reverse proxy

Terminate TLS on the bank's existing web gateway (F5, Citrix ADC, Apache, nginx) and forward to `http://<server>:3000`. If no gateway is available, install nginx on the same server and use the file `deploy/nginx.conf` from the repository.

```

sudo apt-get install -y nginx
sudo cp /opt/sourcing-crm/deploy/nginx.conf /etc/nginx/sites-
available/sourcing-crm
sudo nano /etc/nginx/sites-available/sourcing-crm  # set server_name and the
two certificate paths
sudo ln -s /etc/nginx/sites-available/sourcing-crm /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

```

Whatever proxy is used, it must:

Setting	Value	Why
Upstream	<code>http://127.0.0.1:3000</code> (or the app server's IP on a gateway)	The app itself
HTTP to HTTPS	Redirect port 80 to 443	Sign-in only works on HTTPS
TLS versions	TLS 1.2 and 1.3 only	Bank standard
Forwarded headers	<code>Host</code> , <code>X-Forwarded-Proto: https</code> , <code>X-Forwarded-For</code>	Correct links and audit log IP addresses
Request body limit	8 MB	Bulk upload spreadsheets and Emirates ID photos
Read timeout	60 seconds	Bulk uploads of a few thousand rows
Response headers	<code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: no-referrer</code> , <code>Strict-Transport-Security: max-age=31536000</code>	Prevents framing and leaks

If the app runs on a different host from the gateway, change the Docker port mapping or add a firewall rule so that only the gateway can reach port 3000. Then open `https://crm-test.yourbank.ae` in a browser; the sign-in page should load with a valid padlock.

Step 4: Configuration reference

All settings are environment variables in `/etc/sourcing-crm.env` (systemd) or `deploy/.env` (Docker). The file holds a password, so keep it mode 600 and never commit it. Restart the service after any change.

Variable	Required	Value	Notes
<code>PORT</code>	No	<code>3000</code>	Port the app listens on behind the proxy
<code>DB_FILE</code>	Yes	<code>/var/lib/sourcing-crm/crm.db</code>	SQLite database file. Docker sets <code>/data/crm.db</code> automatically

Variable	Required	Value	Notes
COOKIE_SECURE	Yes	1	Marks the sign-in cookie Secure. Must be 1 behind HTTPS; sign-in fails otherwise
ADMIN_NAME	First start	Name of the first team leader	Used only when the database has no users
ADMIN_EMAIL	First start	Their bank email	Their sign-in name
ADMIN_PASSWORD	First start	A strong temporary password	Change it on the Users page after first sign-in, then blank it in the file
IT_EMAIL	Optional	IT mailbox	Receives approved call-recording requests
IT_EMAIL_WEBHOOK_URL	Optional	URL of an email relay that accepts POST {to, subject, text}	Without it, requests are only shown in the app
TL_WEBHOOK_URL	Optional	Teams or Slack incoming webhook	Alert when a case is marked verification pending
PROCESSING_WEBHOOK_URL	Optional	Teams or Slack incoming webhook	Alert when a scheduled call-back is due
CALL_BOT_URL , CALL_BOT_SECRET	Optional	Leave blank	Calling bot, on hold. Blank hides the feature
PUBLIC_URL	Optional	https://crm-test.yourbank.ae	The address users see, used in alerts and bot links

The app makes outbound HTTPS calls only to the webhook URLs that are set. With all of them blank it makes no outbound connections at all.

Step 5: First sign-in and loading users

IT hands over after step 2 below; the business owner and MIS user do the rest inside the app.

1. **IT:** open `https://crm-test.yourbank.ae`, sign in as `ADMIN_EMAIL` with `ADMIN_PASSWORD`. On the Users page, set a new password for this account.
2. **IT:** on the Users page, create one user with role **MIS** (name, bank email, mobile, WhatsApp number). Pass the temporary password to that person securely, then blank `ADMIN_PASSWORD` in the environment file.
3. **MIS:** sign in, open **Bulk upload → Users**, download the template, fill it in and upload. Load team leaders and sales managers first, then sales staff, so that reporting lines resolve. The upload returns a file of sign-in details to distribute securely.
4. **MIS:** open **Bulk upload → Targets**, download the template, and load targets for the current cycle. Credit card and account targets are case counts; personal loan and auto loan targets are AED disbursal amounts. Cycles run from the 21st to the 20th and are named after the month they end in.
5. **Business owner:** have one person from each role (sales, processing, team leader, sales manager, MIS, business head, governance) sign in and confirm their dashboard loads. Users who forget passwords are reset by a team leader or MIS on the Users page.

To load existing cases from a spreadsheet, MIS uses **Bulk upload → Cases** with its template. Card activation status is loaded later from **Bulk upload → Cards**.

Step 6: Backups, restores, updates and logs

Backups. The whole system state is the one database file. The script `deploy/backup.sh` takes a consistent copy with SQLite's online backup, safe while the app is running, compresses it and deletes copies older than 30 days. Schedule it nightly and ship the copies off the server.

```
# systemd route: run from root's crontab (needs the sqlite3 package)
sudo crontab -e
0 2 * * * DB_FILE=/var/lib/sourcing-crm/crm.db
BACKUP_DIR=/var/backups/sourcing-crm /opt/sourcing-crm/deploy/backup.sh

# Docker route: copy the file out of the volume
docker compose -f /opt/sourcing-crm/deploy/docker-compose.yml exec crm sh -c
"cp /data/crm.db /data/crm-backup.db" && \
  docker cp $(docker compose -f /opt/sourcing-crm/deploy/docker-compose.yml
ps -q crm):/data/crm-backup.db /var/backups/sourcing-crm/crm-$(date +%F).db
```

The backup contains customer identifiers and salaries. Treat the backup location with the same controls as the live database.

Restore. Stop the service, replace the database file with the uncompressed backup, fix ownership, start the service.

```
sudo systemctl stop sourcing-crm
gunzip -c /var/backups/sourcing-crm/crm-20261008-0200.db.gz | sudo tee
/var/lib/sourcing-crm/crm.db > /dev/null
sudo chown crm:crm /var/lib/sourcing-crm/crm.db
sudo systemctl start sourcing-crm
```

Updates. Pull the branch and restart. Database schema changes apply themselves on start; take a backup first.

```
cd /opt/sourcing-crm && sudo git pull
sudo systemctl restart sourcing-crm
# Docker route instead:
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --
build
```

Logs and monitoring.

Need	Command
Live log	<code>journalctl -u sourcing-crm -f</code> or <code>docker compose -f deploy/docker-compose.yml logs -f crm</code>
Is it up	<code>curl -s http://127.0.0.1:3000/api/meta</code> returns JSON; add this to the monitoring tool as an HTTP check on the public URL too
Disk	Alert when the database directory has less than 2 GB free
Service restarts	The unit restarts the app automatically after a crash; alert on repeated restarts in <code>journalctl</code>

The app logs sign-ins, errors and webhook failures to standard output, which journald or Docker captures. Who viewed or revealed which customer identifiers is recorded inside the app under **Access log**, not in the server log.

Step 7: Optional integrations

None of these are needed for the test environment. Each is switched on by setting its variable in Step 4 and restarting.

Integration	What IT provides	What the app does
Teams or Slack alerts to team leaders	An incoming webhook URL for the team leaders' channel, set as <code>TL_WEBHOOK_URL</code>	Posts a message when a case is marked verification pending
Teams or Slack alerts to the processing team	An incoming webhook URL set as <code>PROCESSING_WEBHOOK_URL</code>	Posts a message when a scheduled call-back falls due (checked every 30 seconds)
Email to IT for call-recording requests	An HTTPS endpoint that accepts <code>POST JSON {"to","subject","text"}</code> and sends an email (a small relay in front of the bank SMTP server), set as <code>IT_EMAIL_WEBHOOK_URL</code> , plus the mailbox in <code>IT_EMAIL</code>	Sends an email when governance approves a request to pull a call recording
Calling bot	On hold. Leave <code>CALL_BOT_URL</code> and <code>CALL_BOT_SECRET</code> blank	Feature stays hidden
WhatsApp alerts	Not built. Would need a WhatsApp Business API account	Nothing yet

Webhook calls go out from the app server, so the firewall must allow outbound HTTPS to the Teams, Slack or relay host when one of these is enabled. A failed webhook is logged and never blocks the user's action.

Step 8: Acceptance test checklist

Run this on the real server with real devices before the pilot users are invited. Tick each item; the business owner signs off when all are ticked.

Infrastructure (IT)

- ☐ `https://crm-test.yourbank.ae` loads on a staff laptop and on a staff phone with a valid padlock
- ☐ `http://` redirects to `https://`
- ☐ Port 3000 is not reachable from a staff laptop, only through the proxy

- ☐ Service restarts on its own after `sudo kill -9 <pid>` (or `docker kill`)
- ☐ Nightly backup produced a file in the backup folder, and a restore into a scratch copy opens with `sqlite3 ... "select count(*) from users"`
- ☐ The server clock shows UAE time correctly (`timedatectl`)

Application (business owner and one user per role)

- ☐ First team leader signed in and changed the password
- ☐ Users loaded by bulk upload; each role's dashboard opens
- ☐ Targets loaded for the current cycle; sales staff see their own, team leaders see their team's
- ☐ Scan a real Emirates ID front and back from a phone camera on the New case form; the number and name fill in
- ☐ Type an Emirates ID by hand; dashes appear automatically
- ☐ Voice typing fills a text field, and "Read it back to check" marks a correctly read Emirates ID as valid (Chrome, Edge or Safari; not Firefox)
- ☐ One case through the full flow: sales creates, processing verifies by call, team leader actions the pending case
- ☐ Verification "call back later" set two minutes ahead triggers the call-back alert and, if configured, the Teams or Slack message
- ☐ Changing the product, loan amount, interest rate, FPD or card sourced type after verification sends the case back for re-verification
- ☐ Phone, Emirates ID, passport and salary show masked; reveal works, re-hides after 3 minutes, and the reveal appears in the Access log for governance
- ☐ A screenshot of any page shows the user's name and time watermark
- ☐ Credit card completed cases appear as inactive with ageing; bulk card activation upload marks them active with a date
- ☐ Chat: a case discussion and a direct message deliver, and governance can read them
- ☐ Bulk upload of cases from the template with the bank's own test spreadsheet

Security notes and production go-live

The test environment is suitable for a pilot with a small group and test or consented customer data. The items below are required before the system becomes the system of record, and information security should review them now.

Already built in

- Passwords hashed with scrypt; sessions in HttpOnly, Secure, SameSite cookies; all writes require JSON bodies, which blocks cross-site form posts

- Role-based access: each role sees only its own cases, team or scope
- Customer identifiers and salary masked on screen; every reveal and case view logged with user, time and IP in the Access log, visible to governance and business heads
- Watermark of the viewer's name, code and time on every page
- Process runs as an unprivileged user with a read-only filesystem apart from the data directory; no third-party packages, so no dependency vulnerabilities to patch beyond Node.js itself

Required before production go-live (currently on hold; raise with the business owner to schedule)

Item	Why	Effort
Single sign-on with the bank directory (SAML or OpenID Connect)	Joiners, leavers and password policy follow the bank's	Medium: an identity provider endpoint from IT, then an app change
Login lockout and password rules	Brute-force protection	Small app change
Session timeout of 15 to 30 minutes idle	Shared devices	Small app change
Field-level encryption of Emirates ID and passport numbers at rest	Database file or backup theft exposes only ciphertext	Medium: key kept outside the database, e.g. in the bank's key vault
Penetration test	Bank policy for any system holding customer data	Scheduled with the security team
Log forwarding to the bank SIEM	Central audit	IT: forward journald and the Access log export
Data retention policy	Cases, chat messages and access logs grow indefinitely	Decision from compliance, then a scheduled purge

Keep in mind during the pilot

- Screenshots and phone photos of the screen cannot be blocked by a web application. The watermark and access log are deterrents and evidence, not prevention.
- Speech recognition uses the browser's own engine. On Chrome and Edge the audio goes to Google or Microsoft; nothing is sent to the CRM server. If that is not acceptable, disable the microphone in the browser policy and voice typing will be hidden.

Troubleshooting

Symptom	Cause	Fix
Service exits at once; log says <code>node:sqlite</code> or <code>DatabaseSync</code> is not available	Node.js older than 22.13	Install Node.js 22.13 or newer and restart
Sign-in page loads but signing in returns to the same page	<code>COOKIE_SECURE=1</code> while the site is reached over plain HTTP, or the proxy does not pass <code>X-Forwarded-Proto</code>	Use HTTPS end to end; check the proxy headers in Step 3
Sign in works on HTTP test but not HTTPS	<code>COOKIE_SECURE</code> unset	Set <code>COOKIE_SECURE=1</code> and restart
<code>SQLITE_READONLY</code> or <code>EACCES</code> in the log	Data directory not writable by the <code>crm</code> user, or not listed in <code>ReadWritePaths</code>	<code>chown -R crm:crm /var/lib/sourcing-crm;</code> keep the unit file's <code>ReadWritePaths</code>
<code>EADDRINUSE: 3000</code>	Another process on port 3000	Stop it, or set <code>PORT</code> and the proxy upstream to another port
Bulk upload fails with <code>413</code>	Proxy body limit too small	Set the proxy limit to 8 MB
Emirates ID scanner says it cannot load	Scanner files not installed (systemd route)	Run <code>npm run setup:ocr</code> in <code>/opt/sourcing-crm</code> and restart
Camera or microphone buttons do nothing	Page not on HTTPS, or the browser blocked the device	Check the padlock; allow camera and microphone for the site in the browser
Voice typing missing on a user's browser	Firefox, or a browser policy disabling speech recognition	Use Chrome, Edge or Safari
Teams or Slack alert not received	Webhook URL wrong, or outbound HTTPS blocked from the server	Log shows <code>webhook failed</code> ; test with <code>curl -X POST <url> -d '{"text":"test"}'</code> from the server

Symptom	Cause	Fix
Call-back alert late	Server clock wrong	Enable NTP; alerts are checked every 30 seconds in UAE time
Admin password forgotten and no other team leader	Only the first start creates the admin	Stop the service, move the database file aside, start once with a new <code>ADMIN_PASSWORD</code> , then restore the database and have the original admin reset others, or ask the developer for a reset script
Database file growing fast	Chat messages and access log	Expected; about 1 MB per 1,000 cases plus logs. Plan retention per the security notes

For anything else, send the last 100 lines of `journalctl -u sourcing-crm` (or the Docker log) with the time of the error.